



Модуль 9. Основи продуктивності та оптимізації

Урок 1. Продуктивність як системне мислення: де виникають проблеми і з чого починати

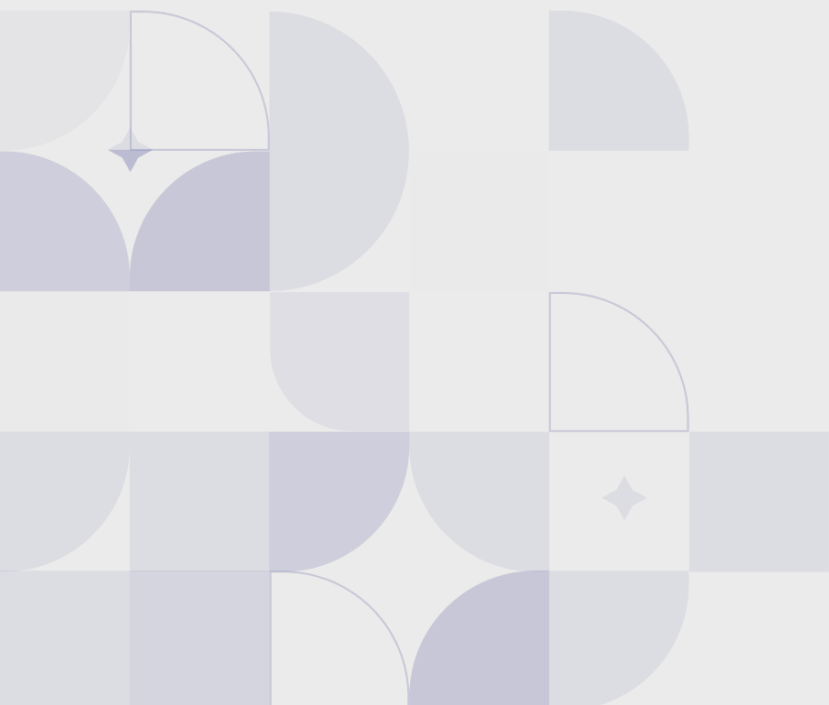
Не “прискорити PHP”, а знайти вузькі місця

Логи/метрики → продуктивність

- Модуль 8 навчив вас бачити, що відбувається в продакшені
- ті ж самі логи/метрики показують, де витрачається час
- Модуль 9 навчить використовувати ці дані для цільової оптимізації

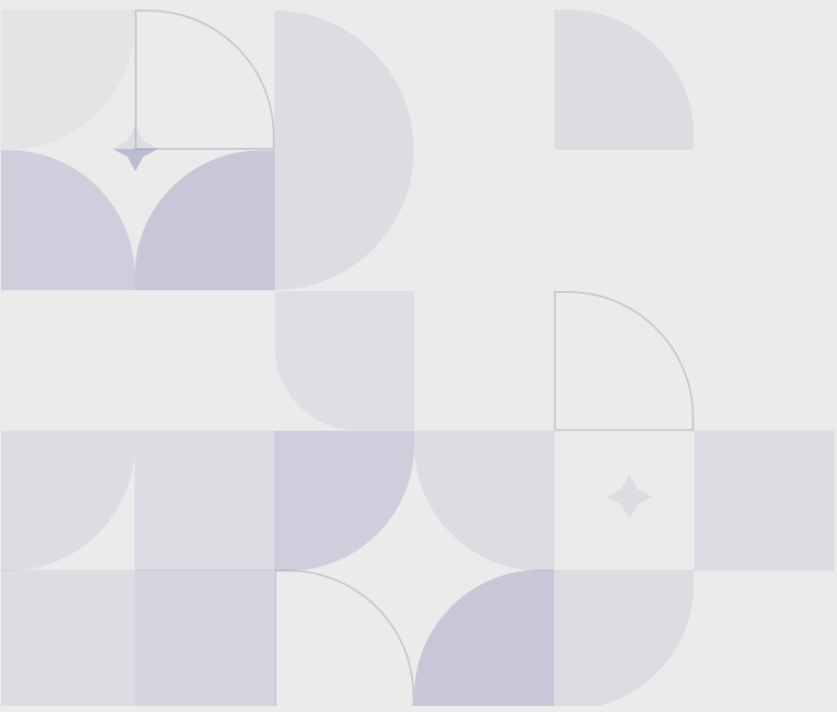
Мета уроку

1. подивитися на продуктивність як на властивість всієї системи, а не функції
2. розібратися, де зазвичай виникають вузькі місця
3. відокремити передчасну оптимізацію від необхідної
4. побачити роль кешування, профайлінгу, БД і API



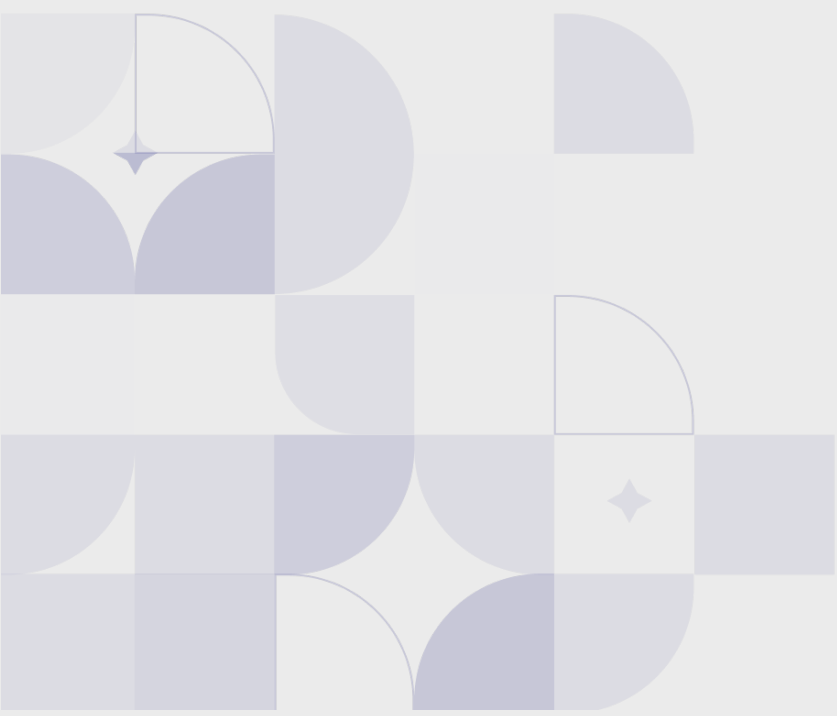
PHP повільний” - зручне, але хибне пояснення

- у більшості веб-додатків вузьке місце не в PHP
- основні проблеми: БД, кеш, N+1, важкі операції в запиті
- звинувачувати мову простіше, ніж аналізувати систему



Швидко, стабільно, передбачувано

- час відповіді (p50/p95/p99)
- стабільність під навантаженням
- масштабованість при зростанні даних (10 → 10 000 платежів)



Як виглядає повільна система

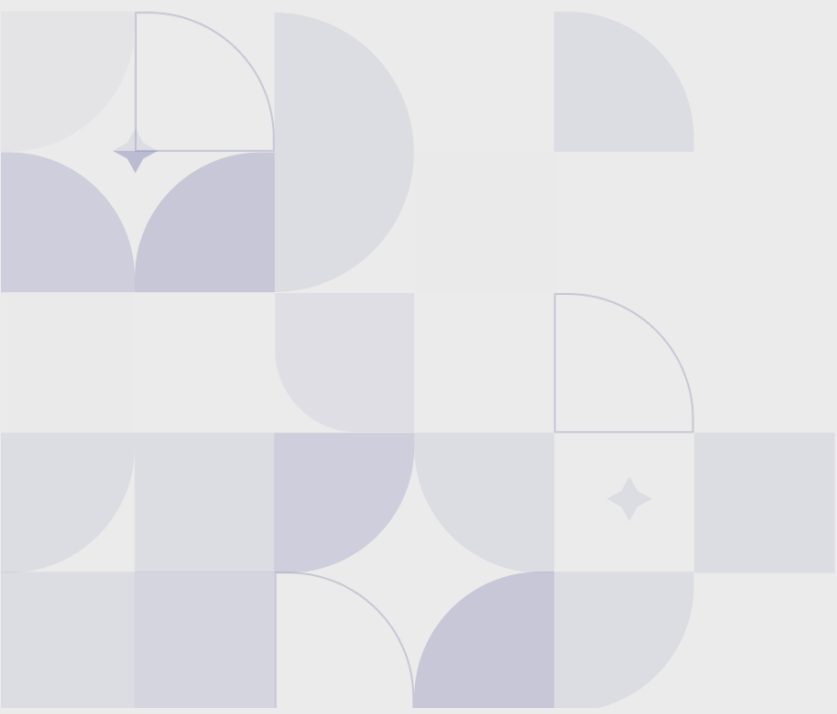
- повільні API-ендпоінти (список платежів, виписки)
- таймаути (клієнт або шлюз кидає запит)
- ріст навантаження на БД; деградація при рості даних («у мене локально швидко»)

ТОП-6 проблем продуктивності

- повільні/масові запити до БД
- відсутність кешу
- N+1; зайві обчислення в запиті
- синхронні важкі операції (звіти, агрегації)
- неефективні відповіді API (без пагінації, “толстый” JSON)

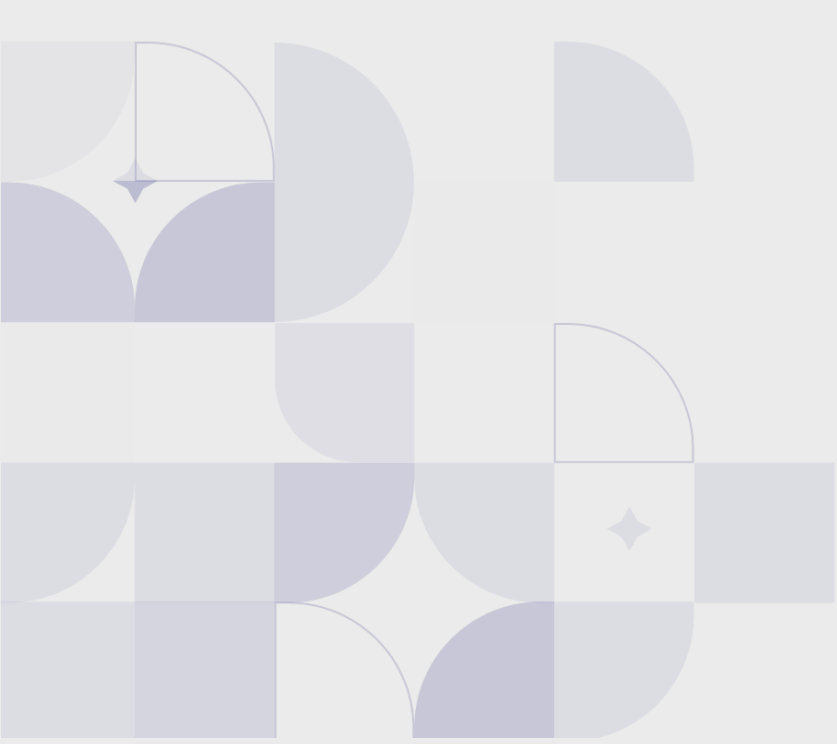
Чому “оптимізувати наперед” небезпечно

- оптимізація без вимірів → витрати часу не там, де треба;
- ускладнення коду без реального виграшу
- правильний порядок: **виміряти → знайти вузьке місце → оптимізувати → знову виміряти**



Практика: дивимось на ендпоінт до оптимізації

Що він робить, звідки бере дані, де може
гальмувати



Кеш - не хак, а інструмент

Визначення:

- зберігання результатів обчислень/запитів у швидкому сховищі
- дані, результати запитів, об'єкти, відповіді API
- тимчасова копія, джерело істини - БД

Зменшуємо навантаження, пришвидшуємо відповіді

- менше запитів до БД
- швидше відповіді (Redis/пам'ять швидші за БД)
- краща стабільність у піки

Баланс між швидкістю й актуальністю

Часто кешують:

- довідники, налаштування, агреговану статистику
- read-only списки, публичні дані

Не варто кешувати:

- баланс рахунку в реальному часі
- критичні персоналізовані значення без чіткої інвалідації

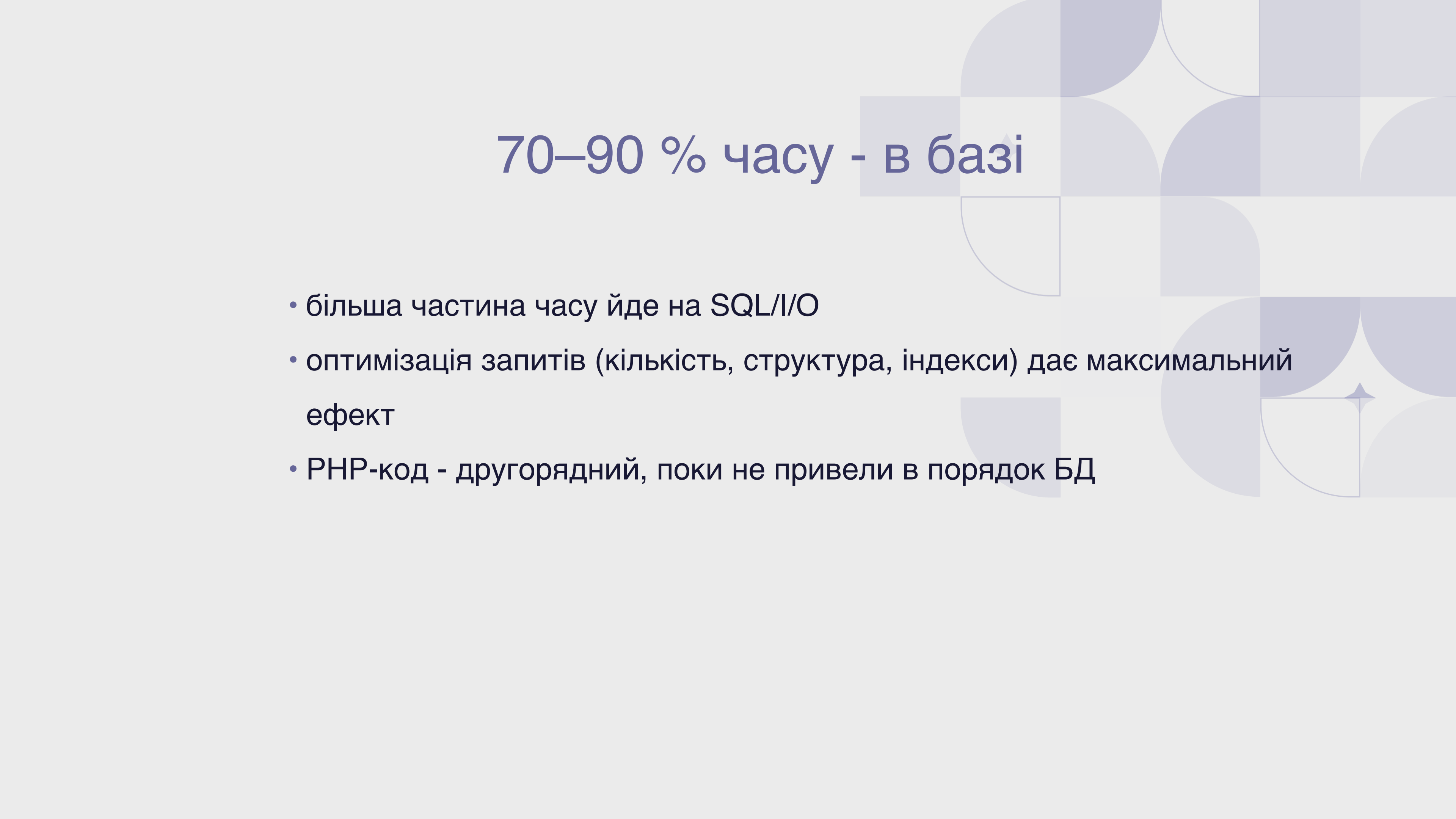
Практика: що кешувати у вашому ендпоінті

Довідники, агрегації, read-only списки

Профайлінг відповідає на “де повільно?”

Інструменти:

- Laravel Telescope, Debugbar, xdebug
- показують час запитів, кількість SQL, пам'ять



70–90 % часу - в базі

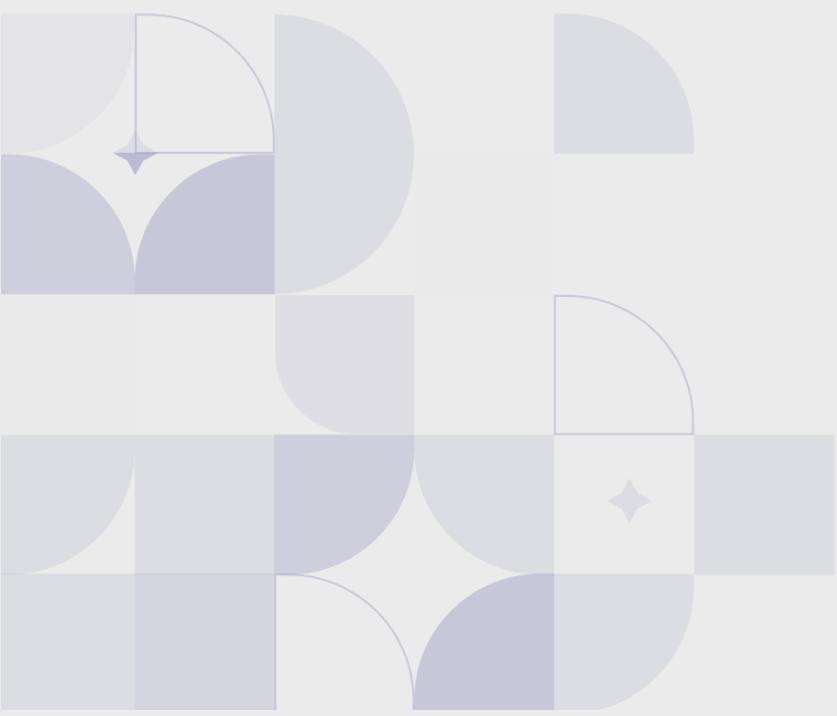
- більша частина часу йде на SQL/I/O
- оптимізація запитів (кількість, структура, індекси) дає максимальний ефект
- PHP-код - другорядний, поки не привели в порядок БД

Чому “1 + N запитів” = біда

- один запит на список, потім по запиту на кожен елемент
- час і навантаження ростуть лінійно з N
- рішення - eager loading / join

Практика: шукаємо потенційний N+1

Списки платежів/рахунків і їх зв'язки



Індекси пришвидшують SELECT, але уповільнюють INSERT/UPDATE

- індекси потрібні на полях із частими WHERE/JOIN/ORDER BY
- занадто мало індексів → повільні SELECT
- занадто багато → повільні записи й зайвий обсяг

Менше даних → швидший API

- не повертати зайві поля
- завжди використовувати пагінацію для списків
- думати про розмір JSON (особливо у фінансових звітах)

Де шукати вузьке місце в GET /accounts/{id}/payments

Потенційні проблеми:

- N+1 за зв'язками
- відсутність пагінації
- відсутність індексів на **account_id/created_at**
- важка логіка в циклі

Практика: обираємо один перший крок оптимізації

Eager loading, пагінація, кеш...

Як легко зробити гірше, думаючи, що робите краще

Приклади:

- оптимізація без вимірів
- кешування всього підряд
- складні рішення «на майбутнє»
- ігнорування БД
- боротьба з симптомами, а не з причиною

Що ми сьогодні зафіксували

- продуктивність — властивість системи, а не лише коду
- основні вузькі місця: БД, кеш, N+1, важка логіка
- кеш/профайлінг/індекси як базові інструменти
- правильний підхід: **вимірювати → оптимізувати → знову вимірювати**

Системне мислення > “мікрооптимізації”

**Справжня оптимізація починається з
вимірювань і розуміння системи; без цього
будь-яке “прискорення” - лотерея**

Дякуємо за увагу

- Питання?
- До наступного заняття спробуйте хоча б виміряти час роботи одного “повільного” ендпоінта у своєму проєкті